SEN4100 – Software Project Management

Week 2 Lecture Notes: Project Lifecycles & Process Models

Course Information & Academic Framework

- **Course Learning Outcomes (CLOs) Addressed:** Understand the fundamentals of software project management from initiation to closure; compare traditional predictive processes (Waterfall, V-Model) with modern agile and lean approaches (Scrum, Kanban); select and justify appropriate project management methodologies for varying software environments.
- **Primary Textbooks:** Pressman & Maxim (*Software Engineering: A Practitioner's Approach*), Hughes & Cotterell (*Software Project Management*), Ian Sommerville (*Software Engineering*).

1. Foundations of Software Process Models

A software development process is a structured framework of activities, actions, tasks, milestones, and work products required to engineer high-quality software within defined temporal and budgetary boundaries. Because software is inherently intangible and malleable, a Software Process Model provides an abstract representation of the engineering lifecycle, establishing governance, traceability, and predictability.

1.1 The Generic Process Framework

Regardless of whether a project follows a rigid predictive workflow or an iterative agile cycle, Roger Pressman defines five universal framework activities that exist across all software engineering endeavors:

1. **Communication:** Collaborative stakeholder engagement to elicit business goals, define technical constraints, and gather user requirements.
2. **Planning:** Estimating effort and financial budgets, scheduling tasks, establishing critical paths, and constructing the Work Breakdown Structure (WBS).
3. **Modeling:** Creating domain abstractions, architectural blueprints (e.g., C4 models, ER diagrams), and behavioral models to guide implementation.
4. **Construction:** Source code compilation, static analysis, automated unit verification, and peer code reviews.
5. **Deployment:** Packaging deliverables into release artifacts, configuring CI/CD deployment pipelines, conducting user acceptance validation, and monitoring production telemetry.

These five primary activities are continuously governed by **Umbrella Activities**, which span the

entire project timeline:

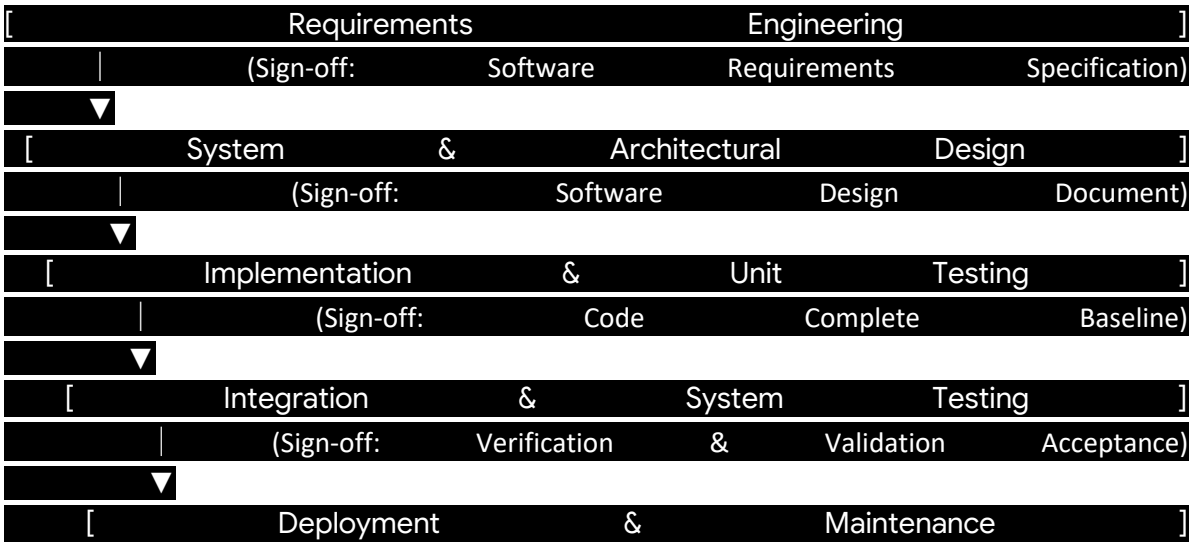
- **Software Quality Assurance (SQA):** Verifying compliance with engineering baselines and standards.
- **Software Configuration Management (SCM):** Tracking revisions, branching models, and release packages.
- **Risk Management:** Continual identification, tracking, and mitigation of technical and operational threats.
- **Measurement & Metrics:** Quantitative performance tracking against schedule and cost baselines.

2. Traditional & Prescriptive Process Models

Prescriptive (plan-driven or predictive) models operate on the assumption that software requirements can be largely understood, captured, and validated up front. They emphasize formal milestone gates, explicit architectural documentation, and disciplined change control boards.

2.1 The Classic Waterfall Model (Royce, 1970)

The Waterfall model organizes development into a strictly sequential, downstream flow. A subsequent phase begins only after the deliverables of the preceding phase are formally reviewed, verified, and baselined.



- **Core Characteristics:**
  - Strict milestone transitions; returning to an earlier phase is treated as an exceptional

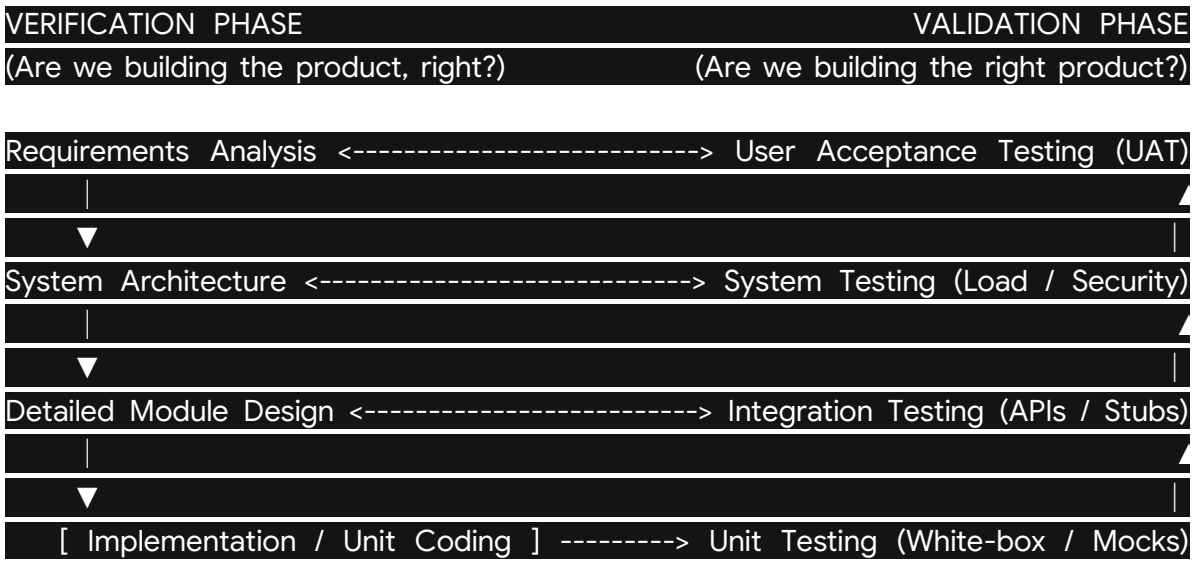
change-control deviation.
- Heavy emphasis on exhaustive specifications (SRS, SDD, Interface Control Documents).
- High managerial visibility through clear percentage-of-phase completion tracking.

- **Advantages:**
- Disciplined, structured, and straightforward for contractual governance and fixed-bid procurement.
- Eliminates ambiguity early when working with deterministic, static specifications.
- Promotes clean architectural separation before physical implementation begins.

- **Pathologies & Failure Modes:**
- **The "Big Bang" Integration Trap:** Working software is not integrated until late in the lifecycle; hidden interface discrepancies and performance bottlenecks surface near the delivery deadline.
- **The Requirements Inflexibility Dilemma:** Real-world clients cannot articulate all requirements accurately in the abstract without interacting with concrete prototypes.
- **Blocking States:** Downstream specialists (e.g., test engineers) remain idle or underutilized during early analysis and design phases.

2.2 The V-Model (Verification and Validation Model)

The V-Model refines the Waterfall paradigm by visually and procedurally linking each construction phase directly to a corresponding testing tier, establishing early quality assurance.



- **Verification vs. Validation:**
- **Verification:** Evaluates whether intermediate work products satisfy the conditions

imposed by the preceding stage (*"Are we building the product, right?"*). It focuses on static code reviews, walkthroughs, and architectural compliance.
    - **Validation:** Assesses whether the executable system meets the operational intent and business requirements of the client (*"Are we building the right product?"*). It involves dynamic test execution and field testing.
- **Early Test Plan Generation:** Test cases are authored symmetrically during design rather than deferred until after coding:
    - User Acceptance Tests are drafted during **Requirements Analysis**.
    - System Integration Tests are formulated during **System Architecture Design**.
    - Unit Tests are written during **Detailed Module Design**.
- **Primary Target Domains:** Regulated, safety-critical embedded systems (e.g., aerospace avionics under DO-178C, automotive ECUs under ISO 26262, medical devices under IEC 62304) where formal verification and defect traceability are mandatory.

2.3 The Prototyping Paradigm

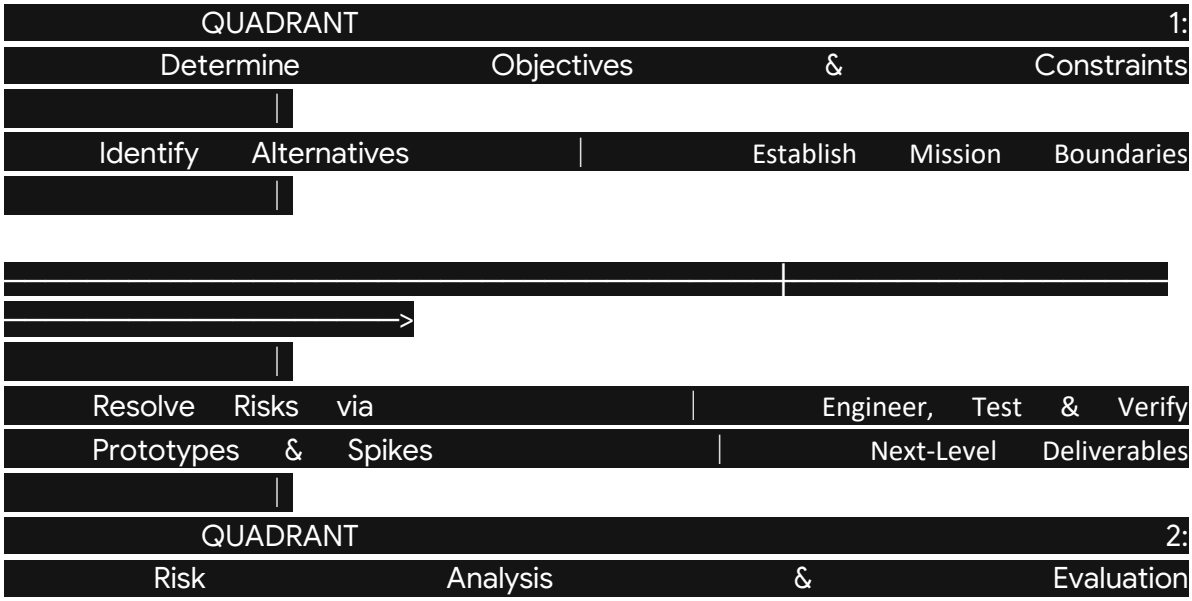
Prototyping mitigates uncertainty by building an operational mock-up or preliminary release to clarify obscure requirements before full-scale development begins.

1. **Throwaway (Rapid) Prototyping:**
    - *Objective:* Resolve specific user interface flows, ambiguous business algorithms, or risky integration points.
    - *Methodology:* Code is constructed rapidly using rapid tooling; once requirements are confirmed, the prototype is discarded, and the production system is engineered cleanly from scratch.
2. **Evolutionary Prototyping:**
    - *Objective:* Deliver a stable, functional core system that incrementally expands with each user feedback cycle.
    - *Methodology:* High engineering standards, modular architecture, and automated test coverage are maintained from day one, transforming the prototype directly into the final product.

- **Managerial Risks:**
    - **The Customer Illusion:** Non-technical stakeholders see a working front-end interface and assume the software is nearly complete, underestimating the time required for security, exception handling, and database optimization.
    - **The "Prototype into Production" Trap:** Management under deadline pressure often mandates that a fragile, throwaway prototype be deployed straight to production, creating severe long-term technical debt.

2.4 The Spiral Model (Barry Boehm, 1988)

The Spiral Model is an iterative, **risk-driven** process framework that combines the structured control of the Waterfall model with the exploration flexibility of prototyping.



- **The Four Quadrants:**

1. **Determine Objectives, Alternatives, and Constraints:** Elicit requirements, identify performance boundaries, and define operational options for the current cycle.
2. **Evaluate Alternatives, Identify and Resolve Risks:** Perform formal risk analysis; execute architectural spikes, simulations, or prototypes to neutralize critical technical threats.
3. **Develop and Verify Next-Level Product:** Construct and test the system increment using an appropriate engineering approach (e.g., Waterfall for stable components, Agile for volatile modules).
4. **Plan Next Phase:** Review progress with stakeholders; decide whether to commit capital to the next spiral loop or terminate the project.

- **Geometric Dynamics:**
  - **Radial Distance (r):** Represents cumulative financial investment and effort expended to date.
  - **Angular Dimension (theta):** Represents progress achieved and milestones validated across successive lifecycle loops.
- **Strategic Value:** Protects organizations from runaway capital expenditure by forcing an explicit risk-assessment milestone before funding each successive iteration.

3. The Agile Paradigm & Foundations

By the late 1990s, the bureaucratic overhead and rigid contracts of heavy predictive methodologies clashed with the fast-moving internet economy. In February 2001, seventeen software practitioners met in Snowbird, Utah, authoring the **Agile Manifesto** to formalize

adaptive software delivery.

3.1 The 4 Core Values of the Agile Manifesto

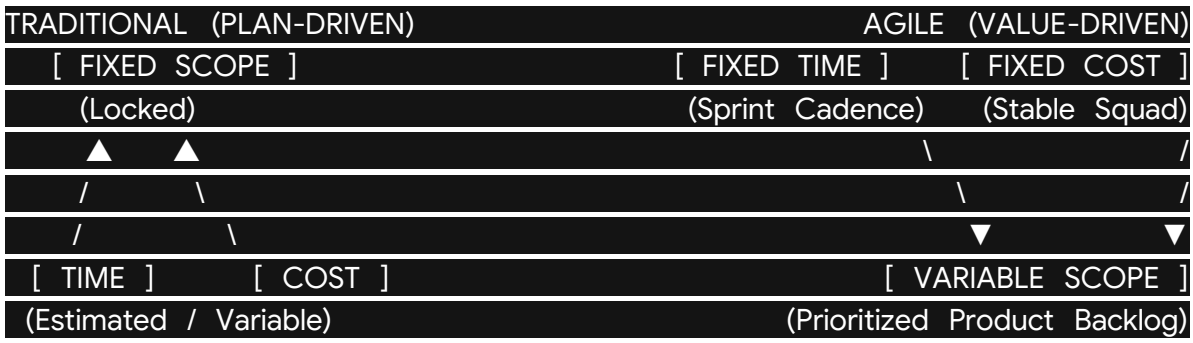
While there is value in the items on the right, Agile values the items on the left more:

Individuals and interactions over processes and tools
Working software over comprehensive documentation
Customer collaboration over contract negotiation
Responding to change over following a plan

1. **Individuals and Interactions over Processes and Tools:** Tooling (e.g., Jira, CI/CD platforms) facilitates work, but high-performing, cross-functional squads with open communication drive real delivery.
2. **Working Software over Comprehensive Documentation:** Specifications are useful, but executable, tested, and deployable software is the only true measure of progress.
3. **Customer Collaboration over Contract Negotiation:** Fixed, adversarial contracts incentivize defensive project management; collaborative partnerships align development teams directly with emerging customer needs.
4. **Responding to Change over Following a Plan:** Rigid adherence to an obsolete initial plan guarantees failure; an adaptive team treats change as a competitive advantage.

3.2 Inverting the Project Management Iron Triangle

The core conceptual difference between traditional plan-driven management and agile value-driven management lies in which project variables are locked versus which are estimated:



- **Traditional Model:** Scope is fixed in contract specifications. Schedule (Time) and Budget (Cost) are mathematically estimated variables. When delays occur, budgets expand or testing phases are compressed, harming quality.

- **Agile Model:** Schedule is locked into short, fixed-length timeboxes (e.g., 2-week Sprints), and Cost is stabilized by funding a dedicated, cross-functional engineering squad. **Scope is the variable parameter**, continually groomed and prioritized by the Product Owner so that the highest-value features ship first.

4. Process Model Comparison and Selection Framework

Choosing an inappropriate lifecycle model is a primary source of project distress. A project manager must evaluate organizational, technical, and regulatory factors before establishing the delivery process.

4.1 Comparative Evaluation Matrix

Evaluation Criterion	Waterfall PDF	V-Model PDF	Spiral Model PDF	Agile (Scrum / Kanban) PDF
Requirements Volatility	Low; completely frozen up front	Low; rigorously baselined	Moderate; explored via prototypes	High; continuously evolving
Primary Project Focus	Orderly, sequential progression	Upfront verification & validation	Systematic risk resolution	Rapid delivery of customer value
Risk Management Profile	Reactive; risks surface late	Moderate; addressed via early test plans	Proactive; risk analysis drives every cycle	Adaptive; mitigated by short feedback loops
Customer Involvement	Limited to initiation and UAT	Limited to analysis and final sign-off	Periodic reviews at spiral gates	Continuous; daily interactions with PO
First Working Software	Near final project deployment	Near final project deployment	Intermediate functional prototypes	End of every iteration / sprint

Cost of Scope Modification	Exponential as project advances	High; necessitates updated test suites	Moderate; factored into subsequent loops	Low; managed via backlog reprioritization
Typical Target Domains	Infrastructure upgrades, legacy re-platforming	Aerospace, medical devices, automotive	High-uncertainty R&D, complex systems	Cloud SaaS, mobile applications, e-commerce

4.2 Systematic Lifecycle Selection Heuristics

To select a software process model systematically, evaluate the following five dimensions:

1. **Requirements Volatility & Clarity:**
   - If requirements are dynamic, emergent, or competitive to **Agile (Scrum / Kanban)**.
   - If specifications are fixed by physical constraints, law, or external standards to **Waterfall or V-Model**.
2. **System Criticality & Failure Impact:**
   - If software failure risks loss of life, severe environmental damage, or existential liability to **V-Model or Spiral Model**.
   - If rapid user acquisition and early time-to-market outweigh minor defects to **Agile**.
3. **Team Co-location & Organizational Culture:**
   - High-trust, cross-functional squads working in shared time zones thrive under **Agile**.
   - Fragmented, multi-vendor enterprise contracts with low trust often demand the documented milestones of **Predictive Models**.
4. **Novelty of Architecture & Technical Debt:**
   - Novel, unproven technologies (e.g., custom AI architectures, cutting-edge distributed databases) require **Spiral cycles or Agile spikes** to address feasibility risk before production construction.
5. **Hybrid Governance ("Water-Scrum-Fall"):**
   - Enterprise environments frequently use hybrid delivery:
     - **Upfront (Waterfall):** High-level business case, regulatory approval, and architectural feasibility.
     - **Middle (Scrum):** Iterative construction, sprint planning, and automated testing across 2-week iterations.
     - **Downstream (Waterfall):** Compliance audits, security reviews, and coordinated client deployment.

5. Industrial Case Studies

Case 1: Firmware for an Intravenous Smart Infusion Pump

- **Context:** Hospital embedded device administering micro-dosages of high-alert medications directly into patient bloodstreams.
- **Appropriate Lifecycle: V-Model (IEC 62304 / FDA Class III Medical Device Compliance)**.
- **Engineering Rationale:** An arithmetic overflow or timing defect can cause fatal consequences. System specifications (dosage algorithms, sensor tolerances, fail-safe protocols) are bound by medical hardware. Every software requirement must maintain bidirectional traceability to its corresponding design element, source code module, and verification test case.

Case 2: Multi-Sided B2C Crypto-Asset Exchange Platform

- **Context:** Financial technology startup launching a consumer trading app facing shifting regulatory guidelines, emerging liquidity mechanisms, and aggressive market competitors.
- **Appropriate Lifecycle: Agile (Scrum with Kanban Continuous Delivery)**.
- **Engineering Rationale:** Requirements volatility is extreme; consumer expectations evolve on weekly cycles. Committing to a 12-month Waterfall plan guarantees launching an obsolete product. The engineering team must ship a Minimum Viable Product (MVP) rapidly, using analytics and production telemetry to guide subsequent sprint backlogs.

6. Alignment of Term Project Deliverables

The concepts covered this week connect directly to your semester team project (20% of your course grade):

- **Milestone 1 Expectation (Week 4 Checkpoint):** Each team of 3–4 students must document their chosen software lifecycle model (e.g., Pure Scrum, Hybrid Water-Scrum-Fall, or Kanban) within their Project Charter and Scope document. You must include a formal technical justification citing requirements, architectural risk, and delivery cadence.
- **Tooling Configuration:** Your squad's tracking setup in Jira or GitHub Projects must match your selected process model (e.g., configured with 2-week sprint iterations and story points for Scrum, or WIP limits and continuous swimlanes for Kanban).

7. In-Class Discussion Scenarios & Review Questions

1. **The Fixed-Price Contract Dilemma:**
   - A client demands a contract with a Fixed Price, a Fixed Delivery Date, and a Fixed Scope, but your engineering team operates using Scrum. What structural project risk does this create? How can a project manager negotiate variable scope or phased releases under a fixed-price umbrella?
2. **The "Zero Documentation" Misconception:**
   - Why is the Agile value *"Working software over comprehensive documentation"* frequently

misunderstood as an excuse to avoid documentation entirely? What constitutes a "lean, high-value" documentation package in an agile software project?

3. **Boehm's Spiral vs. Modern Lean Startup:**
   - How do the risk-reduction cycles of Barry Boehm's 1988 Spiral Model relate conceptually to Eric Ries's modern "Build-Measure-Learn" feedback loop and the Minimum Viable Product (MVP)?

8. Next Week Preview & Assigned Reading

- **Week 3 Topic:** Project Scope, Requirements & Feasibility.
- **Core Agenda:** Authoring the formal Project Charter, creating the Stakeholder Register, conducting technical/economic/operational feasibility studies, establishing the Scope Baseline, and preventing Scope Creep.
- **Preparation Reading:** Pressman & Maxim (Chapters 2 & 3); Hughes & Cotterell (Chapter 2: *Step Wise: An Overview of Project Planning*).